

Informe:

Busqueda clasica vs busqueda adversaria

- En búsqueda clásica hay un solo agente y el entorno no reacciona. Podés calcular toda la secuencia de pasos de antemano y ejecutarla sin cambios en el entorno.
- En la búsqueda adversaria el entorno no es fijo, hay un oponente que actúa en tu contra. No podés planear pasos fijos porque no sabés qué va a hacer él.
- El resultado no es una lista de pasos sino una **estrategia**: un plan que cubre qué hacer ante cada posible respuesta del oponente.

Árboles de juego

- Un **estado** es la foto del tablero en un momento dado: quién puso qué, dónde.
- Las **acciones** son las jugadas válidas que se pueden hacer desde ese estado.
- Los **turnos alternados** significan que un nivel del árbol es tuyo y el siguiente es del oponente, y así se van intercalando.
- **Lutilidades** son valores numéricos que solo aparecen al final del juego: ganaste (+1), perdiste (-1), empataste (0).
- El árbol completo representa todas las partidas posibles que podrían ocurrir.

Minimax como extensión de DFS

- **Valor minimax**: el número que retorna la función recursiva (ej: +1) → dice *qué tan bueno* es un estado

- **Elección de acción:** la *jugada concreta* que lleva a ese valor → se obtiene con `argmax`
- Son cosas distintas: el valor evalúa, el argmax decide
- Ejemplo: minimax dice "desde acá puedo garantizar +1". Argmax dice "la jugada que me lleva ahí es la posición 4"

Límite de escala: complejidad exponencial

- La cantidad de nodos a explorar es $O(b^d)$: `b` = ramificación, `d` = profundidad
- Ta-Te-Ti: $b \approx 9, d \approx 9 \rightarrow \sim 550.000$ nodos → manejable
- Ajedrez: $b \approx 35, d \approx 100 \rightarrow$ más nodos que átomos en el universo → **imposible**
- El problema no es el algoritmo, es que el árbol de juego crece **exponencialmente** con la profundidad
- Minimax puro es exacto pero solo funciona en juegos pequeños

Poda Alfa-Beta, profundidad limitada y heurísticas

Poda Alfa-Beta:

La poda Alfa-Beta es una optimización de Minimax que descarta ramas del árbol que nunca van a cambiar la decisión final, manteniendo dos valores mientras busca: alfa, que representa lo mejor que MAX puede garantizarse hasta ahora, y beta, que representa lo mejor que MIN puede garantizarse. Cuando alfa supera o iguala a beta significa que esa rama nunca sería elegida por ninguno de los dos jugadores, así que se descarta sin explorarla, lo que en la práctica reduce el trabajo de forma drástica. Sin embargo, incluso con esta optimización los juegos complejos siguen siendo demasiado grandes para explorar hasta el final, por eso se combina con profundidad limitada: se corta la búsqueda en un punto arbitrario en vez de llegar a los estados terminales, lo que hace el árbol manejable a costa de perder exactitud. El problema que surge entonces es que en ese punto de corte no hay un valor de utilidad real,

así que se necesita una función heurística que estime qué tan bueno es ese estado intermedio. En ajedrez, por ejemplo, esa función puede mirar la diferencia de piezas, el control del centro del tablero o la seguridad del rey. Son aproximaciones que introducen error, pero sin ellas los juegos reales serían imposibles de jugar para una computadora dentro de un tiempo razonable.

- Si $\alpha \geq \beta \rightarrow$ se poda $\rightarrow$ reduce de $O(b^d)$ a $O(b^{(d/2)})$

MCTS y AlphaGo

- **MCTS** resuelve el problema de no tener una buena heurística: en vez de evaluar un estado con una fórmula, juega miles de partidas aleatorias desde ese estado y mira cuántas gana. Si gana el 70%, ese estado vale 0.7. No necesitas saber *por qué* es bueno, solo verificas empíricamente que desde ahí se gana más seguido.
- Tiene cuatro pasos que se repiten: elegir por dónde bajar en el árbol, agregar un nodo nuevo, jugar una partida aleatoria desde ahí, y subir el resultado actualizando todos los nodos del camino.
- **AlphaGo** tomó MCTS y reemplazó las partidas aleatorias por redes neuronales entrenadas con millones de partidas reales. Una red evalúa qué tan prometedor es un estado, otra sugiere qué jugadas explorar primero.
- El resultado es un híbrido: usa la estructura de búsqueda de MCTS pero guiada por conocimiento aprendido, no por aleatoriedad. Esa combinación es lo que lo hizo imbatible.